

Sviluppo di un sistema di content management leggero basato su Git e tecnologie web moderne

Tesi di Laurea in Informatica

Il Tuo Nome

September 16, 2025

Abstract

Contents

1	Introduzione	3
1.1	Contesto: La Gestione dei Contenuti nel Web Moderno	3
1.2	Obiettivi della Tesi	3
1.3	La Soluzione Proposta	4
1.4	Caso di Studio	4
1.5	Struttura della Tesi	4
2	Analisi delle Tecnologie e Stato dell'Arte	6
2.1	Panoramica delle Soluzioni Esistenti	6
2.1.1	Sistemi di Content Management System (CMS) Tradizionali	6
2.1.2	Generatori di Siti Statici (SSG)	6
2.1.3	Headless CMS	7
2.2	Analisi e Scelta degli Strumenti	7
2.2.1	La Scelta dell'Editor WYSIWYG	7
2.2.2	La Scelta della Piattaforma Backend	8
2.3	Posizionamento del Progetto	8
3	Progettazione e Architettura del Sistema	10
3.1	Architettura Generale	10
3.2	Gestione dei Contenuti Dinamici	10
3.2.1	Disaccoppiamento Contenuto-Presentazione	10
3.2.2	Editing dei Contenuti Online (WYSIWYG)	11
3.3	Integrazione con il Sistema di Version Control (Git)	11
3.3.1	Il Flusso di Salvataggio e Commit Automatico	11
3.3.2	Autenticazione e Sicurezza	12
3.4	Connessione a Servizi Esterni	12
3.4.1	Recupero Dati da Sorgente Esterna	12
3.5	Gestione delle Pagine del Sito	13
4	Dettagli Implementativi	14
4.1	Implementazione del Backend Node.js	14
4.1.1	Gestione delle Richieste e Routing	14
4.1.2	Sistema di Autenticazione con Express-session	15
4.2	Automatizzazione del Versioning su Git	16
4.3	Scraping di Dati Esterni con Cheerio	17
4.4	Personalizzazione del Client SimplyEdit	18
4.4.1	Gestione dello Stato di Autenticazione	18
4.4.2	Override delle Funzionalità di Storage	19

5	Risultati e Caso di Studio	21
5.1	Il Sito Web Risultante	21
5.2	L'Esperienza di Editing	21
5.3	Il Workflow di Versioning in Azione	21
5.4	Ricerca di Pubblicazioni Esterne	21
5.5	Sistema di Autenticazione	21
6	Conclusioni e Sviluppi Futuri	22
6.1	Sintesi del Lavoro Svolto	22
6.2	Valutazione dei Risultati	22
6.3	Limitazioni Attuali	22
6.4	Sviluppi Futuri	22

Chapter 1

Introduzione

1.1 Contesto: La Gestione dei Contenuti nel Web Moderno

La gestione dei contenuti per il web rappresenta da sempre un campo in continua evoluzione, caratterizzato da un ampio spettro di soluzioni tecnologiche. A un estremo si collocano i siti web statici, costruiti con HTML, CSS e JavaScript, che offrono massime prestazioni, elevata sicurezza e semplicità di hosting, ma la cui manutenzione e aggiornamento dei contenuti richiedono necessariamente l'intervento di uno sviluppatore. All'estremo opposto si trovano i Content Management System (CMS) monolitici, come WordPress, Joomla e Drupal. Questi sistemi hanno democratizzato la creazione di siti web, fornendo potenti interfacce di amministrazione che permettono anche a utenti non tecnici di gestire i contenuti in autonomia. Tuttavia, questa facilità d'uso ha un costo: i CMS tradizionali sono spesso architetture complesse che dipendono da uno stack tecnologico specifico (e.g., server PHP, database MySQL), introducendo un significativo overhead in termini di performance, manutenzione e, soprattutto, sicurezza.

Tra questi due estremi esiste uno spazio per soluzioni intermedie, destinate a progetti che necessitano di aggiornamenti dinamici dei contenuti ma che non giustificano la complessità e i costi di manutenzione di un CMS completo. L'esigenza è quella di combinare la leggerezza e la sicurezza dei siti statici con un'esperienza di editing dei contenuti che sia semplice e accessibile.

1.2 Obiettivi della Tesi

L'obiettivo di questo lavoro di tesi è la progettazione e lo sviluppo di un sistema per la gestione di siti web con contenuti dinamici che superi la rigidità dei siti statici senza introdurre la complessità e le dipendenze infrastrutturali di un CMS tradizionale. Per raggiungere tale scopo, il progetto si è posto i seguenti requisiti fondamentali:

- Realizzazione di un sito web con contenuti dinamici senza l'uso di piattaforme CMS complesse come WordPress, Joomla o Drupal.
- Implementazione di un'interfaccia di editing WYSIWYG per la modifica dei contenuti direttamente in pagina.
- Integrazione con servizi REST esterni per l'arricchimento dei contenuti.
- Implementazione di un sistema di version control (Git) integrato che tracci automaticamente tutte le modifiche.
- Sviluppo di funzionalità opzionali come l'integrazione con sistemi esterni per la visualizzazione delle pubblicazioni dei docenti.

1.3 La Soluzione Proposta

Per rispondere ai requisiti delineati, è stata progettata e sviluppata un'architettura basata sull'editor SimplyEdit integrato con un backend leggero basato su Node.js che funge da intermediario con il file system e un repository Git.

Il percorso di sviluppo ha inizialmente esplorato l'utilizzo di TinyMCE come editor WYSIWYG, ma successivamente è emersa la superiorità di SimplyEdit per questo specifico caso d'uso, grazie alla sua architettura più aderente ai requisiti del progetto e alla migliore integrazione con il paradigma di salvataggio basato su Git.

Il cuore del sistema si basa sull'interazione tra questi componenti principali:

1. Un client web moderno, che sfrutta la libreria SimplyEdit per fornire un'esperienza di editing WYSIWYG direttamente sulla pagina, implementando un approccio basato su JSON che permette di disaccoppiare completamente il contenuto dalla sua presentazione.
2. Un server backend basato su Node.js e Express.js, il cui ruolo è fornire le API necessarie al funzionamento del sistema, ricevere i contenuti aggiornati dall'editor, persisterli sul file system e automatizzare il processo di commit sul repository Git locale.
3. Un sistema di integrazione con servizi esterni, implementato attraverso tecniche di web scraping con Cheerio, per arricchire il sito con contenuti dinamici (come le pubblicazioni dei docenti).

Questo approccio trasforma di fatto il repository Git nel vero database del sito, trattando ogni salvataggio come una versione atomica e tracciabile della storia dei contenuti, con tutti i vantaggi che ne derivano in termini di cronologia, recupero di versioni precedenti e potenziale integrazione con pipeline di CI/CD.

1.4 Caso di Studio

Per dimostrare la validità e la flessibilità dell'approccio proposto, il sistema è stato applicato a un caso di studio pratico: la re-implementazione di un sito web ispirato a quello del Dipartimento di Informatica dell'Università di Roma "Tor Vergata". Per la realizzazione dell'interfaccia è stato adottato il template Bootstrap moderno "Nova", adattandone la struttura per renderla compatibile con il sistema di editing dinamico di SimplyEdit.

1.5 Struttura della Tesi

La presente tesi è strutturata come segue:

- Capitolo 2** Analisi delle Tecnologie e Stato dell'Arte: Viene analizzato lo stato dell'arte delle tecnologie per la gestione dei contenuti, confrontando l'approccio proposto con le soluzioni esistenti e giustificando le scelte tecnologiche effettuate, con particolare focus sull'evoluzione dalla valutazione di TinyMCE all'adozione di SimplyEdit come editor principale.
- Capitolo 3** Progettazione e Architettura del Sistema: Viene descritta in dettaglio l'architettura del sistema, illustrando i componenti principali e i flussi di dati, con particolare attenzione all'integrazione di SimplyEdit con il backend Node.js e il sistema di versioning Git.
- Capitolo 4** Dettagli Implementativi: Vengono presentati i dettagli implementativi delle funzionalità più significative, mostrando l'integrazione di SimplyEdit, il sistema di salvataggio automatico su Git, e l'integrazione con i servizi esterni per il recupero delle pubblicazioni.

Capitolo 5 Risultati e Caso di Studio: Vengono mostrati i risultati ottenuti attraverso il caso di studio, con evidenza visiva del sito funzionante, dell'esperienza di editing con SimplyEdit e del workflow di versioning su Git, oltre ai risultati dell'integrazione con i servizi esterni.

Capitolo 6 Conclusioni e Sviluppi Futuri: Vengono tratte le conclusioni, valutando criticamente il lavoro svolto, discutendone le limitazioni e proponendo possibili sviluppi futuri, inclusi eventuali miglioramenti all'integrazione con SimplyEdit.

Chapter 2

Analisi delle Tecnologie e Stato dell'Arte

La scelta di un'architettura per la gestione dei contenuti web richiede un'attenta analisi delle soluzioni esistenti, al fine di comprendere i compromessi di ciascuna e posizionare correttamente il proprio lavoro. Questo capitolo analizza il panorama tecnologico, dalle soluzioni più tradizionali a quelle più moderne, per poi approfondire il processo di valutazione e selezione degli strumenti specifici che ha portato all'architettura finale del progetto.

2.1 Panoramica delle Soluzioni Esistenti

2.1.1 Sistemi di Content Management System (CMS) Tradizionali

I CMS monolitici come WordPress, Joomla e Drupal rappresentano da anni la soluzione dominante per la creazione di siti web dinamici. Il loro punto di forza risiede in un'interfaccia di amministrazione completa e user-friendly, che astrae l'utente da ogni complessità tecnica, e in un vasto ecosistema di plugin e temi che estendono le funzionalità di base.

Tuttavia, questa architettura "all-in-one" presenta svantaggi significativi che li rendono inadatti per il contesto di questa tesi: la dipendenza da uno stack server specifico (tipicamente LAMP/LEMP) e da un database relazionale li rende pesanti, complessi da mantenere e vulnerabili dal punto di vista della sicurezza. La necessità di aggiornamenti costanti sia del core che dei componenti terzi li trasforma in un bersaglio primario per attacchi informatici, richiedendo competenze specifiche per la loro manutenzione.

2.1.2 Generatori di Siti Statici (SSG)

In risposta alla complessità dei CMS tradizionali, sono emersi i Generatori di Siti Statici (SSG) come Jekyll, Hugo, Gatsby e Next.js (in modalità statica). Questi strumenti adottano un approccio fondamentalmente differente: separano la fase di costruzione del sito da quella di hosting. I contenuti, tipicamente scritti in formati semantici come Markdown o YAML, vengono processati staticamente per generare un sito composto unicamente da file HTML, CSS e JavaScript.

I vantaggi sono notevoli in termini di performance, sicurezza (mancando un database o codice server-side dinamico, la superficie di attacco si riduce drasticamente) e semplicità di deployment. Il loro workflow è intrinsecamente basato su Git, favorendo pratiche di sviluppo moderne. Tuttavia, il loro principale limite, nel contesto di questa tesi, è l'assenza nativa di un'interfaccia di editing visuale online: la modifica dei contenuti richiede competenze tecniche (conoscenza di

Git, formati di markup) e un processo manuale di modifica dei file sorgente e successivo commit, risultando poco accessibile per utenti non tecnici.

2.1.3 Headless CMS

L'approccio Headless CMS rappresenta un'evoluzione che tenta di unire i vantaggi dei due mondi precedenti. Sistemi come Contentful, Strapi, TinaCMS o Directus disaccoppiano completamente il backend di gestione dei contenuti (il "corpo") dal frontend di presentazione (la "testa"). I contenuti vengono gestiti tramite un'interfaccia web dedicata e resi disponibili tramite API, tipicamente REST o GraphQL, che il frontend consuma per renderizzare i contenuti.

Questo approccio offre massima flessibilità al frontend, permettendo di utilizzare qualsiasi tecnologia, e centralizza la gestione dei contenuti. Tuttavia, introduce nuove complessità: le soluzioni SaaS (Software as a Service) legano a un fornitore esterno con costi ricorrenti e dipendenze infrastrutturali, mentre quelle self-hosted possono richiedere una configurazione non banale (server, database, manutenzione). Inoltre, il flusso di editing è tipicamente disaccoppiato dalla visualizzazione finale, limitando l'esperienza WYSIWYG.

2.2 Analisi e Scelta degli Strumenti

La selezione degli strumenti per questo progetto è stata un processo iterativo, guidato dai requisiti specifici della traccia e confermato dall'esperienza pratica. Particolare attenzione è stata posta sulla scelta dell'editor WYSIWYG, componente cruciale per l'esperienza utente finale.

2.2.1 La Scelta dell'Editor WYSIWYG

Il percorso di selezione dell'editor è emblematico del processo di ricerca e sviluppo del progetto, dimostrando come requisiti specifici possano guidare verso scelte tecnologiche non ovvie.

Valutazione iniziale: TinyMCE La prima scelta è ricaduta su TinyMCE, un editor JavaScript open-source estremamente potente, maturo e considerato uno standard industriale, utilizzato in progetti di grande rilevanza. La sua integrazione iniziale nel template Bootstrap "Nova" ha dimostrato la sua flessibilità nel rendere editabile qualsiasi elemento del DOM attraverso una configurazione minimale.

Tuttavia, sono emerse rapidamente delle complessità architetturali che lo rendevano non ottimale per questo specifico caso d'uso. TinyMCE è progettato primariamente per operare come un "campo di testo ricco" tradizionale e la sua integrazione in un flusso di salvataggio basato su file system e Git avrebbe richiesto una notevole quantità di codice custom:

- **Identificazione del contenuto:** Sarebbe stato necessario sviluppare una logica complessa per identificare univocamente ogni area modificata e associarla a una porzione specifica del file HTML da salvare.
- **Pulizia dell'HTML:** L'output di TinyMCE, per quanto configurabile, tende a inserire classi e attributi specifici dell'editor (ad esempio per la gestione dello stile), che avrebbero dovuto essere "puliti" lato server prima di ogni salvataggio per non "inquinare" il codice sorgente del template.
- **Complessità lato client:** La gestione di più aree editabili indipendenti sulla stessa pagina avrebbe richiesto la configurazione e la gestione di multiple istanze dell'editor, aumentando significativamente la complessità del codice JavaScript lato client.

La scelta finale: SimplyEdit Durante questa fase di valutazione, è emersa la libreria SimplyEdit. Questo strumento, sebbene meno noto di TinyMCE, si è rivelato più allineato alla filosofia del progetto per diverse ragioni fondamentali.

A differenza di TinyMCE, SimplyEdit non si concepisce come un editor da "embeddare" in campi specifici, ma come uno strato applicativo che si sovrappone all'intera pagina. La sua filosofia "HTML-first", basata sull'aggiunta di semplici attributi `data-simply-*` direttamente nel codice HTML, ha permesso di definire le aree e le liste editabili in modo dichiarativo e con una quantità minima di JavaScript configurabile.

Il vantaggio decisivo è stata la sua architettura di storage intrinsecamente flessibile. SimplyEdit è progettato per essere agnostico rispetto al sistema di salvataggio sottostante e permette di sovrascrivere facilmente il suo storage layer di default. Questa caratteristica si è rivelata il gan- cio fondamentale che ha permesso di intercettare l'evento di salvataggio e di reindirizzarlo a un endpoint custom sul server Node.js, abilitando così l'integrazione diretta e pulita con il workflow di commit su Git, senza dover manipolare pesantemente l'output dell'editor.

2.2.2 La Scelta della Piattaforma Backend

Per il backend, la scelta è ricaduta su Node.js con il framework Express.js. Questa piattaforma si è rivelata ideale per il ruolo di "server leggero" richiesto dal progetto, offrendo diversi vantaggi chiave:

- **Architettura a eventi non bloccante:** La natura asincrona di Node.js è perfetta per gestire operazioni di I/O come la scrittura di file sul disco, senza bloccare il thread principale e mantenendo alta la responsività del server.
- **Gestione efficiente di processi esterni:** La facilità con cui Node.js, tramite il modulo `child_process`, può eseguire comandi di sistema è stata essenziale per l'integrazione trasparente con Git, permettendo di eseguire commit automatici in modo programmatico.
- **Ecosistema NPM:** L'accesso al vasto registro di pacchetti di NPM ha semplificato notevolmente l'implementazione di funzionalità avanzate come la gestione delle sessioni utente (`express-session`), il parsing HTML per lo scraping (`cheerio`), e il routing avanzato.
- **Isomorfismo JavaScript:** L'utilizzo di JavaScript sia lato client che lato server ha favorito una maggiore coerenza dello stack tecnologico e ha semplificato il processo di sviluppo.

2.3 Posizionamento del Progetto

Il sistema sviluppato in questa tesi occupa una posizione distintiva nel panorama delle soluzioni di content management, combinando strategicamente elementi di diverse categorie per creare un approccio ibrido che risponde in modo mirato a tutti i requisiti del progetto.

A differenza dei CMS tradizionali, la soluzione proposta è leggera, non richiede un database dedicato e minimizza la superficie di attacco. Contrariamente agli SSG puri, offre un'esperienza di editing visuale in tempo reale senza richiedere competenze tecniche agli editor dei contenuti. Rispetto ai headless CMS, è interamente self-hosted, non dipende da servizi esterni a pagamento e mantiene il contenuto strettamente accoppiato con la presentazione durante la fase di editing, garantendo una vera esperienza WYSIWYG.

La scelta di utilizzare Git non solo come sistema di versioning ma come strato di persistenza principale rappresenta un elemento particolarmente innovativo, che traccia un ponte ideale tra le best practices dello sviluppo software moderno e le esigenze pratiche della gestione contenuti.

Questo approccio trasforma di fatto ogni modifica in un evento versionato e tracciabile, abilitando potenzialmente workflow avanzati di revisione, approvazione e deployment continuo.

Il progetto dimostra quindi come sia possibile creare un sistema di gestione contenuti dinamico, user-friendly e al tempo stesso tecnologicamente semplice, sicuro e mantenibile, colmando efficacemente il gap tra la rigidità dei siti statici e la complessità dei CMS tradizionali.

Chapter 3

Progettazione e Architettura del Sistema

Questo capitolo descrive l'architettura del sistema sviluppato, illustrando come i diversi componenti interagiscono per soddisfare i requisiti di un CMS leggero basato su Git. L'architettura è stata progettata per essere modulare, sicura e facilmente estensibile, seguendo il principio di separazione delle concerni tra interfaccia utente, logica di business e persistenza dei dati.

3.1 Architettura Generale

Il sistema segue un'architettura client-server con i seguenti componenti principali:

- **Frontend:** Costituito da pagine HTML standard, fogli di stile CSS e codice JavaScript. Il componente chiave è la libreria `SimplyEdit`, che si occupa di rendere le sezioni della pagina modificabili e di gestire l'interfaccia utente per l'editing. Il frontend è responsabile della presentazione dei contenuti e dell'esperienza di editing.
- **Backend:** Implementato in Node.js con il framework `Express.js`, è responsabile della gestione delle richieste, della persistenza dei dati, dell'integrazione con Git e della connessione ai servizi esterni.
- **File System:** Utilizzato per la persistenza dei contenuti (file `data.json`) e dei template HTML.
- **Repository Git:** Utilizzato come sistema di versioning per tracciare tutte le modifiche ai contenuti.

L'interazione tra questi componenti è orchestrata tramite chiamate API REST. Il flusso dei dati è disaccoppiato dalla presentazione: i contenuti testuali e strutturali sono conservati in un file `data.json`, mentre i file `.html` agiscono come template di visualizzazione. La Figura ?? illustra il diagramma dell'architettura e le interazioni tra i componenti.

3.2 Gestione dei Contenuti Dinamici

3.2.1 Disaccoppiamento Contenuto-Presentazione

Il requisito di un sito "dinamico" è stato soddisfatto adottando un'architettura di contenuti disaccoppiata, tipica degli approcci Headless CMS. Invece di avere i contenuti cablati all'interno dei

file HTML, essi vengono centralizzati in un unico file `data.json`. Questo file agisce come un mini-database e contiene tutti i testi, i link e i riferimenti alle immagini del sito, organizzati per pagina.

SimplyEdit legge questo file al caricamento della pagina e popola dinamicamente i template HTML. Questo approccio offre due vantaggi principali:

1. **Manutenibilità:** Le modifiche strutturali o di layout al sito richiedono di intervenire solo sui file `.html`, senza toccare i contenuti.
2. **Riutilizzabilità:** Un singolo dato (es. il titolo di una pagina) può essere definito una sola volta nel `data.json` e riutilizzato in più punti del sito (es. nel titolo della pagina, nell'header e nel menu di navigazione), garantendo coerenza.

3.2.2 Editing dei Contenuti Online (WYSIWYG)

L'editing visuale è demandato a SimplyEdit. Tramite l'uso di attributi HTML5 `data-*`, si definiscono le aree della pagina che possono essere modificate:

- `data-simple-field="nomeCampo"`: designa un singolo elemento (un titolo, un paragrafo, un'immagine) come campo editabile.
- `data-simple-list="nomeLista"`: designa un'area che può contenere una lista di elementi ripetibili (es. una lista di "topic" o di membri dello staff). Al suo interno, un tag `<template>` definisce la struttura HTML di ogni singolo elemento della lista.

Quando l'utente modifica un contenuto, SimplyEdit aggiorna la sua rappresentazione interna dello stato dei dati. Al momento del salvataggio, l'intera struttura dati aggiornata viene serializzata in formato JSON e inviata al backend attraverso una richiesta API.

3.3 Integrazione con il Sistema di Version Control (Git)

Questo è il cuore architetturale del progetto, che risponde al requisito di tracciabilità e versioning.

3.3.1 Il Flusso di Salvataggio e Commit Automatico

Il sistema è stato progettato per trattare ogni salvataggio come un commit atomico nel repository Git. Il flusso dettagliato è il seguente:

1. L'utente, dopo aver effettuato le modifiche, preme il pulsante "Save" nella toolbar di SimplyEdit.
2. Il client invia una richiesta `PUT` all'endpoint `/data/data.json` del server Node.js. Il corpo della richiesta contiene la versione aggiornata e completa del file `data.json`.
3. Il server riceve la richiesta e, dopo aver validato la struttura JSON e sanitizzato i contenuti per prevenire attacchi XSS, sovrascrive il file `data/data.json` presente sul file system.
4. Immediatamente dopo il successo della scrittura su disco, il server, tramite il modulo `child_process` di Node.js, esegue in sequenza i comandi:

```
git add data/data.json
git commit -m "Update content - [TIMESTAMP]"
```

5. Solo al termine di queste operazioni, il server risponde al client con un messaggio di successo. In caso di errore in qualsiasi fase, il server invia una risposta di errore appropriata.

Questo garantisce che il repository Git sia sempre un riflesso fedele e aggiornato dei contenuti del sito, creando una cronologia completa di ogni modifica.

3.3.2 Autenticazione e Sicurezza

Per evitare che chiunque possa modificare i contenuti e generare commit, l'accesso alle API di scrittura è stato protetto. È stato implementato un sistema di sessioni tramite il middleware `express-session`:

- Un utente deve prima autenticarsi tramite una pagina di login che invia una richiesta `POST` a un endpoint `/login`.
- Se le credenziali sono corrette (attualmente verificate tramite un semplice controllo `hard-coded` per scopi dimostrativi), il server crea una sessione per l'utente.
- Tutte le rotte "sensibili" (come `PUT /data/data.json` e le rotte per la creazione di pagine) sono protette da un middleware `isAuthenticated` che controlla l'esistenza di una sessione valida prima di consentire l'esecuzione dell'operazione.
- La sessione ha una scadenza predefinita e viene invalidata al `logout` esplicito dell'utente.

3.4 Connessione a Servizi Esterni

Per soddisfare il requisito di integrazione con servizi esterni, è stata implementata la funzionalità di ricerca delle pubblicazioni. L'architettura scelta è quella di un proxy API:

3.4.1 Recupero Dati da Sorgente Esterna

1. Il client non contatta direttamente il sito esterno (`art.torvergata.it`), per evitare problemi legati alle policy `CORS` (Cross-Origin Resource Sharing) del browser.
2. Viene invece esposto un endpoint sul nostro server `Node.js`: `GET /api/search-publications`.
3. Quando questo endpoint viene chiamato dal client, è il server stesso che si fa carico di interrogare il sito esterno tramite una richiesta `HTTP`.
4. Il server recupera la pagina `HTML` dei risultati, la processa utilizzando la libreria `Cheerio` (un'implementazione di `jQuery` per il server) per effettuare lo `scraping`, ed estrae le informazioni rilevanti (titolo, autori, link).
5. Infine, il server formatta i dati estratti in un array di oggetti `JSON` e li restituisce al client, che si occuperà solo di visualizzarli.

Questa architettura a proxy centralizza la logica di `scraping` lato server, rendendo il client più leggero e evitando problemi di sicurezza legati alle restrizioni `CORS`.

3.5 Gestione delle Pagine del Sito

Oltre alla modifica dei contenuti, il sistema permette la creazione di nuove pagine. Anche questa funzionalità è gestita tramite endpoint dedicati sul server (POST `/create-blank-page` e POST `/create-page-from-template`), protetti da autenticazione.

Il processo di creazione di una nuova pagina segue questo flusso:

1. L'utente seleziona l'opzione di creazione pagina dall'interfaccia di SimplyEdit.
2. Il client, tramite codice JavaScript custom, intercetta questa azione e invia una richiesta all'endpoint appropriato sul server.
3. Il server crea il nuovo file HTML:
 - Per pagine vuote: genera un file HTML minimale con la struttura base e le aree editabili predefinite.
 - Per pagine da template: copia il template specificato e lo personalizza in base ai parametri ricevuti.
4. Il server aggiorna il file di navigazione (se necessario) per includere la nuova pagina nel menu.
5. Il server esegue un commit Git per registrare la creazione della nuova pagina.
6. Il client reindirizza l'utente alla nuova pagina creata.

Questa funzionalità dimostra come il sistema possa essere esteso per supportare operazioni più complesse oltre alla semplice modifica dei contenuti, mantenendo sempre il tracciamento delle modifiche attraverso Git.

Chapter 4

Dettagli Implementativi

Questo capitolo analizza in dettaglio le soluzioni tecniche e gli algoritmi adottati per implementare le funzionalità chiave del sistema. Verranno presentati e commentati degli estratti di codice significativi, focalizzandosi sulle parti più rilevanti del backend Node.js e della logica custom del client.

4.1 Implementazione del Backend Node.js

4.1.1 Gestione delle Richieste e Routing

Il server è stato implementato utilizzando Express.js, che facilita la definizione delle rotte e la gestione delle richieste HTTP. Di seguito la configurazione base del server:

```
const express = require('express');
const session = require('express-session');
const app = express();
const port = 3000;

// Middleware per il parsing del corpo delle richieste
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true }));

// Configurazione delle sessioni
app.use(session({
  secret: 'my-secret-key',
  resave: false,
  saveUninitialized: false,
  cookie: { secure: false, maxAge: 24 * 60 * 60 * 1000 } // 24 ore
}));

// Servizio dei file statici
app.use(express.static('.'));

// Definizione delle rotte API
app.put('/data/data.json', isAuthenticated, handleDataSave);
app.get('/api/search-publications', handlePublicationSearch);
app.post('/login', handleLogin);
app.post('/logout', handleLogout);
```

```

app.get('/check-auth', handleCheckAuth);
app.post('/create-blank-page', isAuthenticated, handleCreateBlankPage);
app.post('/create-page-from-template', isAuthenticated, handleCreateFromTemplate);

// Avvio del server
app.listen(port, () => {
  console.log(`Server in esecuzione su http://localhost:${port}`);
});

```

4.1.2 Sistema di Autenticazione con Express-session

La sicurezza delle operazioni di scrittura è garantita da un sistema di sessioni basato sul pacchetto `express-session`. Un middleware, denominato `isAuthenticated`, viene anteposto a tutte le rotte che modificano il file system.

```

// Middleware di protezione
const isAuthenticated = (req, res, next) => {
  if (req.session.isAuthenticated) {
    return next(); // Utente autenticato, prosegui
  }
  res.status(401).json({ error: 'Accesso negato. Effettuare il login.' });
};

// Gestione del login
app.post('/login', (req, res) => {
  const { username, password } = req.body;

  // Verifica semplificata delle credenziali
  if (username === 'admin' && password === 'password') {
    req.session.isAuthenticated = true;
    req.session.username = username;
    res.json({ success: true, message: 'Login effettuato con successo' });
  } else {
    res.status(401).json({ success: false, error: 'Credenziali non valide' });
  }
});

// Gestione del logout
app.post('/logout', isAuthenticated, (req, res) => {
  req.session.destroy((err) => {
    if (err) {
      console.error('Errore durante il logout:', err);
      return res.status(500).json({ error: 'Errore durante il logout' });
    }
    res.clearCookie('connect.sid');
    res.json({ success: true, message: 'Logout effettuato con successo' });
  });
});

// Verifica stato autenticazione

```

```
app.get('/check-auth', (req, res) => {
  res.json({ isAuthenticated: !!req.session.isAuthenticated });
});
```

Questo semplice ma efficace meccanismo assicura che solo gli utenti che hanno completato con successo il login possano accedere alle funzionalità di modifica.

4.2 Automatizzazione del Versioning su Git

Il requisito fondamentale di integrare un sistema di version control è stato risolto automatizzando i comandi Git direttamente dal server Node.js ad ogni operazione di salvataggio.

```
const { exec } = require('child_process');
const fs = require('fs').promises;

// Gestione salvataggio dati e commit automatico
app.put('/data/data.json', isAuthenticated, async (req, res) => {
  const filePath = './data/data.json';
  const content = JSON.stringify(req.body, null, 2); // Formattazione leggibile

  try {
    // Scrittura del file
    await fs.writeFile(filePath, content, 'utf8');
    console.log('File data.json salvato con successo.');
```

```
    // Esecuzione comandi Git
    const timestamp = new Date().toISOString();
    const commitMessage = `Aggiornamento contenuti - ${timestamp}`;

    exec(`git add "${filePath}"`, (addError, addStdout, addStderr) => {
      if (addError) {
        console.error('Errore git add:', addStderr);
        return res.status(500).json({ error: 'File salvato ma errore in git add' });
      }

      exec(`git commit -m "${commitMessage}"`,
        (commitErr, commitStdout, commitStderr) => {
          if (commitErr) {
            console.error('Errore git commit:', commitStderr);
            return res.status(500).json({
              error: 'File salvato ma errore in git commit'
            });
          }

          console.log('Commit Git eseguito:', commitStdout);
          res.json({
            success: true,
            message: 'File salvato e commit eseguito con successo'
          });
        }
      );
    });
  } catch (err) {
    console.error('Errore salvataggio:', err);
    return res.status(500).json({ error: 'Errore salvataggio' });
  }
});
```

```

        });
    });
});
} catch (error) {
    console.error('Errore scrittura file:', error);
    res.status(500).json({ error: 'Errore durante il salvataggio del file' });
}
});

```

Come si evince dal codice, dopo aver scritto con successo il file `data.json`, viene invocata la funzione `exec` dal modulo `child_process` per eseguire i comandi Git. Questo approccio rende il versioning una parte intrinseca del processo di salvataggio.

4.3 Scraping di Dati Esterni con Cheerio

Per implementare la ricerca di pubblicazioni senza avere accesso a un'API ufficiale, si è ricorso a una tecnica di web scraping lato server.

```

const cheerio = require('cheerio');

// Ricerca pubblicazioni da art.torvergata.it
app.get('/api/search-publications', async (req, res) => {
    const { query } = req.query;

    if (!query || query.trim().length < 3) {
        return res.status(400).json({
            error: 'La query di ricerca deve contenere almeno 3 caratteri'
        });
    }

    try {
        const searchUrl =
            `https://art.torvergata.it/search?q=${encodeURIComponent(query)}`;

        const response = await fetch(searchUrl);
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        const html = await response.text();
        const $ = cheerio.load(html);

        const publications = [];

        // Estrazione dati dalla tabella dei risultati
        $('#tableView_body table tbody tr').each((i, row) => {
            const columns = $(row).find('td');
            if (columns.length >= 3) {
                const titleElement = $(columns[1]).find('a');
            }
        });
    }
});

```

```

const title = titleElement.text().trim();
const link = `https://art.torvergata.it${titleElement.attr('href')}`;
const authors = $(columns[2]).text().trim();
const year = $(columns[3]).text().trim();

if (title) {
  publications.push({
    title,
    link,
    authors,
    year,
    source: 'art.torvergata.it'
  });
}
});

res.json({
  success: true,
  count: publications.length,
  results: publications
});
} catch (error) {
  console.error('Errore durante lo scraping:', error);
  res.status(500).json({
    error: 'Errore durante la ricerca delle pubblicazioni'
  });
}
});

```

La libreria Cheerio si è rivelata fondamentale: essa fornisce un'API simile a jQuery per navigare e manipolare il DOM in ambiente server. Il codice estrae i dati dalla tabella dei risultati e costruisce un array di oggetti JSON puliti.

4.4 Personalizzazione del Client SimplyEdit

Per integrare le funzionalità del backend con l'interfaccia utente, è stato necessario estendere e personalizzare il comportamento di default di SimplyEdit.

4.4.1 Gestione dello Stato di Autenticazione

```

// simply-edit-config.js - Gestione autenticazione
document.addEventListener('DOMContentLoaded', function() {
  // Verifica stato autenticazione
  function checkAuth() {
    return fetch('/check-auth')
      .then(response => response.json())
      .then(data => {
        const isAuthenticated = data.isAuthenticated;

```

```

// Mostra/nascondi pulsante modifica
const editButton = document.getElementById('simply-edit-toggle');
if (editButton) {
    editButton.style.display = isAuthenticated ? 'block' : 'none';
}

// Redirect se non autenticato ma si tenta di accedere
if (!isAuthenticated && window.location.hash === '#simply-edit') {
    window.location.href = '/login.html';
    return false;
}

return isAuthenticated;
})
.catch(error => {
    console.error('Errore verifica autenticazione:', error);
    return false;
});
}

// Verifica iniziale
checkAuth();

// Verifica periodica (ogni 5 minuti)
setInterval(checkAuth, 5 * 60 * 1000);
});

```

4.4.2 Override delle Funzionalità di Storage

```

// simply-edit-config.js - Override salvataggio pagine
document.addEventListener('simply-storage-init', function() {
    if (!window.editor) return;

    // Sovrascrive la funzione di salvataggio pagine
    editor.storage.page.save = function(url, html) {
        return new Promise((resolve, reject) => {
            let newPagePath = new URL(url, window.location.origin).pathname;

            // Determina il tipo di creazione (pagina vuota o da template)
            const isBlankPage = !html;
            const endpoint = isBlankPage ?
                '/create-blank-page' : '/create-page-from-template';

            fetch(endpoint, {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    path: newPagePath,
                    html: html
                })
            })
                .then(response => {
                    if (!response.ok) reject(response);
                    resolve(response.json());
                })
                .catch(reject);
        });
    };
});

```

```

    })
  })
  .then(response => response.json())
  .then(data => {
    if (data.success) {
      // Reindirizza alla nuova pagina in modalità modifica
      window.location.href = `${newPagePath}#simply-edit`;
      resolve();
    } else {
      console.error('Errore creazione pagina:', data.error);
      reject(new Error(data.error || 'Errore sconosciuto'));
    }
  })
  .catch(error => {
    console.error('Errore richiesta creazione pagina:', error);
    reject(error);
  });
});
};
});

```

Questo dimostra come l'architettura flessibile di SimplyEdit abbia permesso di integrare le sue funzionalità di base con la logica custom del nostro backend.

Chapter 5

Risultati e Caso di Studio

5.1 Il Sito Web Risultante

5.2 L'Esperienza di Editing

5.3 Il Workflow di Versioning in Azione

5.4 Ricerca di Pubblicazioni Esterne

5.5 Sistema di Autenticazione

Chapter 6

Conclusioni e Sviluppi Futuri

6.1 Sintesi del Lavoro Svolto

6.2 Valutazione dei Risultati

6.3 Limitazioni Attuali

6.4 Sviluppi Futuri

Bibliography